SQA__Week1_Task

Internee Name: Mahzaib Fatima

DevelopersHub Corporation

Software Quality Assurance (SQA) – 3-Week Project Task

Project Title: Bug Tracking and Test Automation for a Web Application
Submission Deadline: 3rd April, 2026

Week 1 – Manual Testing and Bug Reporting

Objective:

To understand the fundamentals of software testing and gain practical experience in manual testing, test case design, and bug reporting.

Learning Focus:

- Software Development Life Cycle (SDLC) and Software Testing Life Cycle (STLC)
- Functional and User Interface Testing
- Test Case Writing Techniques
- Bug Identification and Reporting

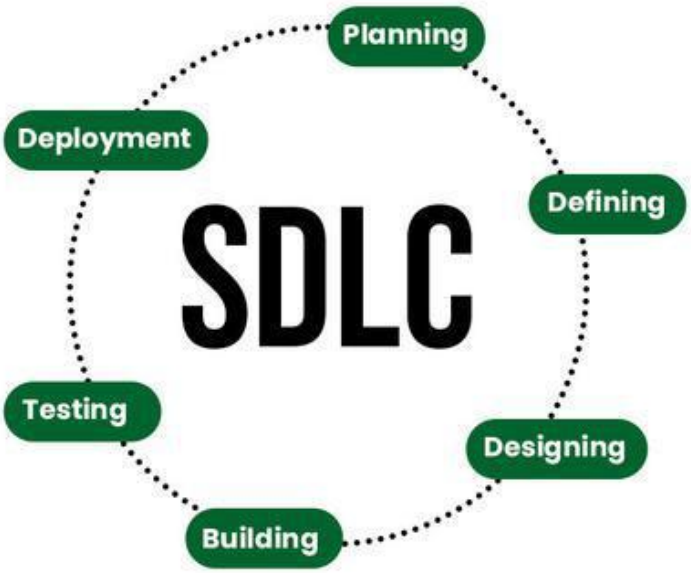
Study SDLC and STLC, focusing on the role of QA in the software development process.

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

The main phases of SDLC are Requirement Analysis, Design, Development, Testing, Deployment, and Maintenance.

The purpose of SDLC is to deliver high-quality software that meets user requirements, is completed on time, and is easy to maintain."



Planning and Requirement Analysis

- This is the first and most important phase of SDLC, where the team defines what to build and why.
- In this phase, feasibility studies, cost estimation, scheduling, and resource planning are done.
- The output includes a project plan and feasibility report, prepared by project managers, senior engineers, and stakeholders.

Defining Requirements (SRS)

- In this phase, detailed functional and non-functional requirements are gathered and documented in the SRS (Software Requirement Specification) document.
- The SRS / CSR acts as a guideline for the development team and ensures everyone understands what to build.
- Business Analysts and Product Owners are the key people involved in this phase.

Designing Architecture

- In this phase, the system requirements are converted into a technical blueprint that defines the system structure and technology stack.
- It includes High-Level Design (HLD) for overall architecture and Low-Level Design (LLD) for component logic and database details.
- The output is a Design Document Specification (DDS), prepared by system architects and lead developers.

Development (Coding)

- This is the phase where developers write the actual code based on the design documents.
- Activities include coding, code reviews, unit testing, and static code analysis using tools like IDEs and Git.
- The output is source code and executable software, developed by frontend, backend, and full-stack developers.

Testing (Quality Assurance)

- In this phase, the QA team tests the software to find bugs before release. It includes Unit Testing, Integration Testing, System Testing, and User
- Acceptance Testing (UAT).
- The output is bug reports, test cases, and quality reports, prepared by QA engineers and testers.

Deployment & Maintenance.

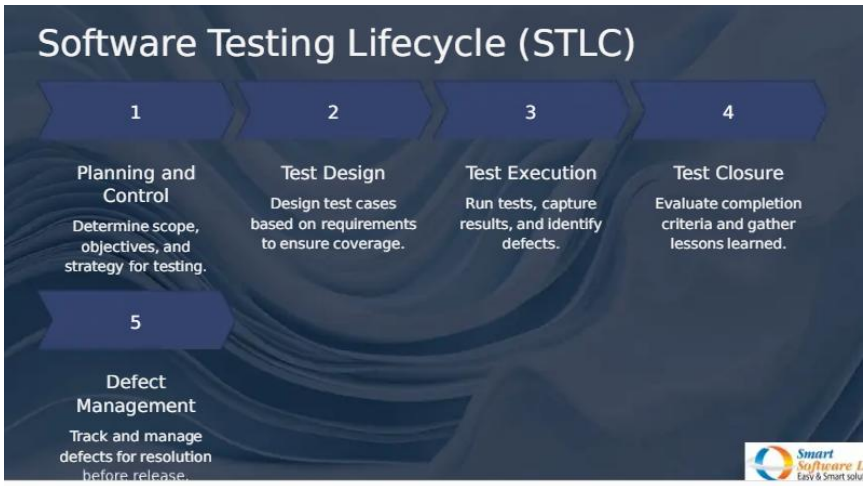
- Deployment is the phase where the software is released to end-users, often using automated CI/CD pipelines. It includes setting up production

environments, deploying code, and performing smoke testing, and the output is a live application managed by DevOps engineers and release managers.

- Maintenance is the continuous phase after deployment, where bugs are fixed, performance is improved, libraries are upgraded, and new features are added. The output includes patches, updates, and new versions, handled by support engineers and developers.

Importance of SQA in SDLC Software Quality Assurance (SQA) plays a critical role in the Software Development Life Cycle (SDLC) by ensuring that software meets required standards and works as intended. This presentation explores the principles of quality assurance and outlines the software testing lifecycle.

STLC (Software Testing Life Cycle)



STLC focuses only on the testing activities in software development.

Phases of STLC

1. Requirement Analysis
    - QA team studies requirements to understand what needs to be tested.
2. Test Planning
    - Test strategy, tools, resources, and schedule are decided.
3. Test Case Design
    - Test cases and test scenarios are written.
4. Test Environment Setup
    - Testing environment is prepared (browser, server, database).

5. Test Execution
    - Test cases are executed.
    - Bugs or defects are reported.
6. Test Closure (Final Phase)
    - Test reports are created.
    - QA evaluates the testing process.
    - Activities: Prepare test summary report, ensure all defects are closed, clean up environment, archive data, conduct lessons learned session. Output: Test Summary Report and archived deliverables.

Role of QA in the Software Development Process

QA (Quality Assurance) ensures the software works correctly and meets the requirements.

Responsibilities of QA

1. Understand Requirements
    - QA studies the system requirements carefully.
2. Create Test Cases
    - Write test cases to check different features of the application.
3. Perform Testing
    - Execute manual or automated tests.
4. Identify Bugs
    - Find errors in the system.
5. Report Bugs
    - Report issues using tools like Jira, Trello, or Bug Tracking Systems.
6. Verify Bug Fixes
    - After developers fix bugs, QA tests again to confirm the fix.
7. Ensure Quality
    - Make sure the final product is stable, secure, and user friendly.

You typically treat this as a two-part task:

(1) systematically find issues

(2) document them in a consistent template.

1. Clarify issue types

- Functional issues: Features not working as expected (e.g., Add to Cart does nothing, search ignores input, broken form validation).
- UI issues: Visual or interaction problems that do not change the business logic (e.g., misaligned elements, overlapping text, inconsistent button styles, poor responsiveness).
- UX/usability issues: Users struggle to complete tasks due to unclear flows, confusing labels, or inconsistent patterns, even if functions technically.

2. How to identify functional issues

- Define expected behavior from specs, user stories, or design docs, then test each scenario and compare actual vs expected.
- Perform positive and negative tests: valid inputs, invalid inputs, edge cases, and error handling.
- Test end-to-end flows (e.g., sign up → login → main action → logout) to catch cross-feature failures.
- Do cross-browser and cross-device checks where relevant to catch functionality that fails only in some environments.

3. How to identify UI/UX issues

- Compare implementation to the design system: spacing, alignment, fonts, colors, iconography, component states (hover, focus, disabled).
- Check responsiveness: different screen sizes, zoom levels, and orientations for overlapping elements, cut-off content, or broken layouts.
- Run quick heuristic and usability checks: consistency of controls, clarity of labels, visibility of feedback/validation, error message clarity.
- Use analytics/feedback: rage clicks, high drop-off points, frequent support tickets to locate confusing or broken UI areas.

4. Documentation template to use

Use the same structure for both functional and UI bugs; just label the type:

- Title: Short, specific (e.g., “Mobile navbar hamburger not clickable on iOS Safari”).
- Type: Functional / UI / UX (and optionally Performance, Security, etc.).
- Environment: Device, OS, browser, app version, build number.
- Preconditions: Any setup needed (user role, existing data, feature flags).
- Steps to reproduce: Numbered, minimal, exact input values and navigation steps.
- Expected result: What should happen according to spec/design.
- Actual result: What actually happens, including error messages or visual problems.
- Evidence: Screenshots, screen recordings, console logs, network errors.
- Severity and priority: How badly it affects users and how urgent the fix.
- Additional notes: Workarounds, related tickets, or hypotheses about cause.

Example bug entry (UI issue)

- Title: “Login button text overlaps icon on 320px width.”
- Type: UI
- Environment: Chrome 123, Android 12, 320×640 viewport
- Steps:
    - Open /login,
    - Resize to 320px width
    - Observe primary button.
- Expected: Icon and label appear side by side with proper spacing.
- Actual: Label text overlaps the icon and becomes unreadable.
- Evidence: Screenshot attached.
- Severity: Medium (feature usable but visually broken).

5. Lightweight checklist

When going through a screen/flow, quickly scan:

- Functional: Do all buttons, forms, links, and API-backed actions work as intended, including error cases.

- Visual: Any misalignment, inconsistent fonts/colors, clipping, overlapping, or broken states (hover, focus, disabled)
- Responsiveness: Does it behave correctly on small/large screens and different browsers.
- Usability: Is the next step obvious, feedback visible, and terminology consistent with the rest of the product.

Test Case 1

Component	Details
Test ID	FB-01
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with email and password
Preconditions	➢ Open Facebook login page ➢ Enter valid email ➢ Enter correct password ➢ Click Log In
Test Data	Username: fatimamahzaib779@gmail Password: .....
Expected Result	User should login successfully and homaepage open
Actual Result	Login successful
Status	Pass

Test Case 2

Component	Details
Test ID	FB-02
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with invalid password
Preconditions	➢ Open login page ➢ Enter valid email ➢ Enter wrong password ➢ Click Login
Test Data	Username: fatimamahzaib779@gmail.com Password: 123456
Expected Result	System should display error message
Actual Result	Error message displayed. “The password you've entered is incorrect.”
Status	Pass

Test Case 3

Component	Details
Test ID	FB-03
Description	Verify login with invalid email
Site	Meta Platforms
Feature under test	Facebook Login page
Preconditions	➢ Open login page ➢ Enter wrong username ➢ Enter password ➢ Click Login
Test Data	Username: wrongemail Password: .....
Expected Result	System should display error message
Actual Result	Error message displayed “The email address or mobile number you entered isn't connected to an account.”
Status	Pass

Test Case 4

Component	Details
Test ID	FB-04
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with empty fields
Preconditions	➢ Open login page ➢ Leave username empty ➢ Leave password empty ➢ Click Login
Test Data	Username: blank Password: blank
Expected Result	Validation or error message should appear
Actual Result	System shows message “The email address or mobile number you entered isn't connected to an account.”
Comment	Incorrect message error display there must shown like “Required email and password”
Status	Fail

Test Case 5

Component	Details
Test ID	FB-05
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify password field is masking
Preconditions	➢ Open login page ➢ Type password in password field
Test Data	Password: 1245@Zaib
Expected Result	Password should appear hidden with dots
Actual Result	Password hidden with dots
Status	Pass

Test Case 6

Component	Details
Test ID	FB-06
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with only email entered
Preconditions	➢ Open Facebook login page ➢ Enter valid email ➢ Click Log In
Test Data	Username: fatimamahzaib779@gmail
Expected Result	Password required message should display
Actual Result	Message displayed “The password you've entered is incorrect”
Status	Fail
Comment	Field validation not specific

Test Case 7

Component	Details
Test ID	FB-07
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with only password entered
Preconditions	➢ Open Facebook login page ➢ Enter valid password only ➢ Click Log In
Test Data	Password: .....
Expected Result	Email required message should display
Actual Result	Message displayed “The email address or mobile number you entered isn't connected to an account”
Status	Fail
Comment	Email Field validation missing

Test Case 8

Component	Details
Test ID	FB-08
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login button functionality
Preconditions	➢ Open Facebook login page ➢ Enter valid Credentials ➢ Click Log In
Test Data	Valid credentials
Expected Result	Button should be clickable and System should process login
Actual Result	Login successful, button working correctly
Status	Pass

Test Case 9

Component	Details
Test ID	FB-09
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify Forgot Password link
Preconditions	➢ Open Facebook login page ➢ Click Forgot Password link
Test Data	none
Expected Result	System should open Password recovery page or recovery page
Actual Result	“Find Your Account” page opened asking for email or mobile number.
Status	Pass

Test Case 10

Component	Details
Test ID	FB-10
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login with spaces in email field
Preconditions	➢ Open Facebook login page ➢ Enter Spaces in Email field ➢ Click login
Test Data	Spaces
Expected Result	Validation message
Actual Result	Error message shown
Status	Fail
comment	Spaces should be restricted

Test Case 11

Component	Details
Test ID	FB-11
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify email field input
Preconditions	➢ Open Facebook login page ➢ Enter Text in Email field
Test Data	Email: gystydwdbsj457
Expected Result	Validation message or Email should appear in field
Actual Result	Text entered successfully
Status	Pass

Test Case 12

Component	Details
Test ID	FB-12
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify login after multiple failed attempts
Preconditions	➢ Open Facebook login page ➢ Enter wrong password several times
Test Data	Email: wrongpassword
Expected Result	Warning or security check
Actual Result	No Security Alert shown (by default)(without two-factor Authentication
Status	Pass
Comment	System need to be security alert by default

Test Case 12

Component	Details
Test ID	FB-12
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify Create New Account button
Preconditions	➢ Open Facebook login page ➢ Click Create New Account
Test Data	None
Expected Result	Registration form should open
Actual Result	Registration form opened
Status	Pass

Test Case 13

Component	Details
Test ID	FB-13
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify the design of the login page is according to figma UI.
Preconditions	➢ Open Facebook login page
Test Data	Use the figma design of login page
Expected Result	The design of the login page is the same figma UI.
Actual Result	The design of the login page and Figma UI is the same
Status	Pass

Test Case 14

Component	Details
Test ID	FB-14
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify system behavior when very long email is entered
Preconditions	➢ Open Facebook login page ➢ Enter very long characters ➢ Click Log In
Test Data	Email: very long email (characters) Password: any value
Expected Result	System should restrict input or show validation error
Actual Result	System accepts long email and shows login error
Status	Fail
Comment	Email length validation missing

Test Case 15

Component	Details
Test ID	FB-15
Site	Meta Platforms
Feature under test	Facebook Login page
Description	Verify system behavior when very long password is entered
Preconditions	➢ Open Facebook login page ➢ Enter valid email ➢ Enter very long password ➢ Click Log In
Test Data	Email: valid email Password: long characters
Expected Result	System should restrict password length or show validation
Actual Result	System accepts password and shows login error
Status	Fail
Comment	Password length validation missing

Bug Identification and Reporting

Open

	Description	Severity	Priority	Steps to	Status
2	Incorrect error message when fields are empty	Medium	High	1. Open Facebook login page 2. Leave	Open
3	Very long email accepted without validation	Low	Medium	Enter 200+ character email and click login	Open
4	Very long password accepted without validation	Low	Medium	Enter long password and click login	Open
5	Generic error when only email is entered	Medium	Medium	Enter email only and click login	Open
6	Generic error when only password is entered	Medium	Medium	Enter password only and click login	Open
7	Login button allows multiple rapid clicks	Low	Low	Click login button multiple times quickly	Open
8	Spaces allowed in email field	Low	Medium	Enter spaces in email field and click login	Open
9					
10					